Trabalho Prático 1 – Indexação Geoespacial e Geometria Computacional Híbrida

Eduardo Birchal e Luis Duarte

October 21, 2026

`https://github.com/DuarteDvv/ifeed`

1 Introdução

O trabalho tem como objetivo o estudo e aplicação de técnicas de geometria computacional e manipulação de sequências, incluindo como essas duas áreas podem auxiliar uma à outra. A atividade em questão envolve a implementação de uma aplicação web cujo usuário interage com um mapa da cidade de Belo Horizonte, podendo visualizar frotas de entregadores baseado em sua proximidade a um restaurante escolhido. A aplicação conta, também, com a capacidade de gerar novos entregadores em localizações realísticas de acordo com uma distribuição estatística, quando instruído pelo usuário. O processo e metodologia envolvidos serão explorados nesse documento.

2 Metodologia

A seguir, serão discutidas as decisões de projeto tomadas em cada etapa da aplicação.

2.1 Pré-Processamento

2.1.1 Coordenadas Geográficas

As coordenadas dos restaurantes foram obtidas usando o cadastro de endereços da Prefeitura de Belo Horizonte[1], cruzadas localmente com o endereço de cada restaurante. Após a normalização dos nomes (incluindo acentos, maiúsculas, pontuação, espaços, abreviações de logradouros e números de imóveis), os restaurantes eram representados como índices em um `defaultdict(set)`, usando endereços como chaves. Cada registro corresponde a um conjunto de coordenadas (x, y), evitando coordenadas duplicatas.

Alguns restaurantes possuíam múltiplas localizações registradas; nesses casos, usou-se o algoritmo de Weiszfeld[2] para obter a mediana geométrica das coordenadas, representando a localização do restaurante como esta. Registros municipais sem geometria, com rua ou número inválido, e outros valores que não podiam ser representados no mapa, foram descartados. Ao total, 20.541 dos 20.859 registros foram mantidos para representação no site.

2.1.2 Grafo de Belo Horizonte

A malha geográfica da cidade foi obtida usando a ferramenta OpenStreetMap, que permite a obtenção de grafos representando a geografia de uma cidade escolhida. Já que a função `graph_from_place` foi usada com o parâmetro `simplify=True`, vértices intermediários foram simplificados através da remoção de todos os nós que não sejam interseções ou fins de via, criando uma aresta diretamente entre os pontos extremos. A geometria completa das arestas originais é salva como um novo atributo `geometry` na nova aresta.

O tempo de viagem foi calculado usando a função `add_edge_travel_times()` da biblioteca OSMnx, que converte o comprimento e velocidade máxima de uma via em uma estimativa de tempo de trajeto pela via. O arquivo final é salvo no formato de um arquivo `graphml`, que é carregado na hora de rodar o site, evitando que o pré-processamento deva ser feito toda vez que a página é aberta.

2.1.3 Segmentação das Ruas

Como as coordenadas geradas pela amostragem Normal, na geração dos entregadores, podem não concidir com o espaço transitável das ruas, ocorre uma etapa de ajuste à malha. A determinação exata da rua mais próxima exigiria calcular a distância do ponto em relação a todas as arestas do grafo viário, resultando em um custo linear em relação ao tamanho total da malha. Para viabilizar essa operação em grandes escalas, emprega-se uma busca heurística indexada por uma KD-tree de duas dimensões.

Para gerar essa KD-tree, cada aresta do grafo é submetida a um particionamento recursivo em seu ponto médio. Se o comprimento de um segmento resultante for superior ao limiar fixado em K metros, o segmento é bipartido novamente, repetindo até que todos os segmentos gerados tenham comprimento menor ou igual a K. Os pontos médios que dividem essas partições são registrados junto com o identificador da aresta original e inseridos em uma KD-tree. O valor de K usado no projeto foi 50 metros.

2.2 Representação do Mapa

2.2.1 Geohashing

A indexação espacial do sistema baseia-se na codificação geohash, uma técnica de particionamento do espaço que transforma pares de coordenadas contínuas de latitude e longitude em cadeias de texto discretas. O espaço é bipartido sucessivamente, iniciando pelo eixo longitudinal e alternando entre os eixos cartesianos. A cada iteração, atribui-se o bit 0 para a metade inferior do intervalo delimitador e o bit 1 para a metade superior. A sequência binária gerada é agrupada em blocos de cinco bits, sendo cada bloco mapeado para um caractere de um alfabeto Base32.

As posições geográficas de restaurantes e entregadores são convertidas em cadeias de oito caracteres, o que fornece precisão de menos de 1 metro na representação pontual. A propriedade fundamental desse método reside na correspondência entre proximidade espacial e compartilhamento de prefixos alfanuméricos, de modo que regiões próximas compartilham os caracteres iniciais da cadeia codificada.

A conversão de coordenadas geográficas em Geohash possui complexidade de tempo e espaço $O(m)$, em que m é o comprimento do código em caracteres. Como cada caractere tem exatamente 5 bits, o algoritmo executa uma busca binária alternando entre os intervalos de latitude e longitude

ao longo de exatamente $5 \times m$ iterações (que fazem apenas o cálculo de ponto médio, comparação e deslocamento de bits em $O(1)$). Uma vez que m é constante, a conversão opera na prática em tempo $O(1)$ e espaço $O(1)$.

2.2.2 Armazenamento e Busca

Para armazenar e consultar essas strings de forma eficiente em memória, o sistema implementa uma trie compacta. Diferentemente de uma trie convencional que reserva um nó para cada caractere da string, a trie compacta funde sequências de caracteres em um único segmento de texto por nó. Cada nó da estrutura contém uma string que define o segmento, uma lista de identificadores numéricos, e uma tabela de dispersão de nós filhos indexada pelo primeiro caractere de cada segmento derivado, garantindo tempo médio constante de localização da transição de ramo.

Na operação de inserção, a chave recebe um caractere delimitador ausente no vocabulário Base32 para sinalizar o término da string, permitindo que prefixos próprios coexistam com chaves completas. Quando ocorre divergência entre uma nova chave e o segmento de um nó existente, o segmento é particionado a partir do maior prefixo comum: cria-se um nó intermediário com o prefixo compartilhado e ramificam-se dois nós descendentes contendo os sufixos restantes.

A recuperação das entidades é realizada pela consulta de prefixos na trie. Dado um restaurante, seu geohash é truncado para os 5-6 primeiros caracteres (dependendo do tamanho do raio). Para evitar perdas de candidatos provocadas por descontinuidades nas bordas da malha do geohash, o sistema calcula os limites da célula central e computa os prefixos de suas 8 células vizinhas. Para cada um dos 9 prefixos, a árvore é percorrida da raiz até o nó correspondente ao término da sequência de caracteres pesquisada, a partir do qual uma rotina recursiva desce por todas as subárvores e coleta os identificadores presentes nos nós filhos. A união dos identificadores é utilizada para resgatar, em tempo constante, os objetos completos de entregadores armazenados em tabelas na memória primária, viabilizando que eles possam ser filtrados por raio euclidiano e que o cálculo de rotas possa ser feito.

2.3 Geração de entregadores

2.3.1 Distribuição espacial

A população de entregadores é gerada em torno dos estabelecimentos cadastrados. A quantidade total de entregadores a serem instanciados é dividida entre os estabelecimentos por meio de uma partição equilibrada, na qual possíveis unidades restantes são distribuídas aleatoriamente sem reposição entre os estabelecimentos. Para cada entregador atribuído a um estabelecimento, as coordenadas geográficas iniciais são amostradas a partir de uma distribuição Normal bivariada centrada na latitude e longitude do local, que é parametrizada por um desvio-padrão em metros. Os valores amostrados nos eixos de latitude e longitude são limitados aos intervalos válidos do sistema de coordenadas terrestres.

2.3.2 Restrição viária

Como mencionado na seção 2.1.3, para ajustar a posição de um entregador, a busca pela rua mais próxima é feita heuristicamente, procurando pelo ponto divisor mais próximo na KD-tree. Como a árvore é construída de forma balanceada, sua altura é limitada por $\lfloor \log M \rfloor$, onde M é o número de pontos divisores. Durante a busca, o algoritmo poda ramos inteiros cuja distância ao

ponto alvo seja maior que a melhor distância encontrada até o momento, consumindo $O(\log M)$ de pilha de recursão em memória. A complexidade, portanto, para descobrir a rua mais próxima de um entregador é $O(\log M)$. Sendo assim, para o conjunto de N entregadores gerados, o custo total de descobrir a rua mais próxima para todos os entregadores é $O(N \log M)$ no pior caso.

Tendo identificado o ponto divisor, a geometria da aresta à qual ele pertence é recuperada, e as coordenadas originais do entregador são projetadas ortogonalmente sobre o segmento correspondente dessa aresta. As coordenadas resultantes da projeção tornam-se a posição definitiva do entregador, a partir da qual é computado o seu geohash.

2.4 Busca em Raio

A busca em raio é iniciada pelo identificador do restaurante e por um raio de atendimento especificado em metros ou quilômetros. O geohash do restaurante é utilizado para consultar na trie compacta os identificadores de entregadores contidos na célula central e nas oito células vizinhas, com precisão que varia entre 5 e 6 dependendo do raio. Os identificadores obtidos são deduplicados e para recuperar as instâncias dos entregadores candidatos na tabela de entregadores.

Para encontrar os códigos geohash das células vizinhas, o sistema faz operações simples sobre as coordenadas da célula central: dada uma célula central C cujo centro tem coordenadas c, encontra-se sua altura h e largura w. As duas células adjacentes horizontalmente a C tem seus centros em $c - w$ e $c + w$, respectivamente. O mesmo vale para as células adjacentes verticalmente, com $c - h$ e $c + h$ compondo seus centros. As células adjacentes diagonalmente estão deslocadas em h e w ambos, com 4 combinações possíveis de positivo e negativo entre esses dois valores. Após o cálculo trivial dessas 8 coordenadas de centro (em $O(1)$, por serem exatamente 8 adições/subtrações), essas coordenadas são convertidas em códigos geohash para que a busca seja feita.

Considerando que m é o tamanho do prefixo e k é a quantidade de entregadores encontrados, a complexidade da etapa de encontrar os entregadores candidatos baseado no seu geohash pode ser descoberta da seguinte forma: para cada um dos 9 prefixos de tamanho m, a descida na trie consome no máximo $O(m)$ comparações de caracteres. Ao alcançar o nó raiz do prefixo, a coleta recursiva de IDs realiza uma DFS sobre a subárvore. Como em uma trie compacta todo nó intermediário possui 2 ou mais filhos, o número de nós internos é sempre menor que o número de folhas, limitando o total de nós visitados a menos de $2k_i$ (onde k_i é o número de entregadores localizados na i-ésima célula). Dessa forma, a travessia e a deduplicação em tabela hash operam em tempo proporcional à soma de candidatos recuperados na grade 3×3, resultando em um custo assintótico de $\sum_{i=1}^{9} O(m + k_i) + O(k) = O(m + k)$ em tempo e espaço auxiliar.

Para cada entregador recuperado, calcula-se a distância até as coordenadas do restaurante por meio da fórmula de Haversine sobre a curvatura da Terra. A distância calculada é comparada ao raio de corte da consulta, registrando-se, para cada candidato, um atributo `within_range`. O sistema preserva todos os candidatos da célula, mesmo os que não pertencem ao raio, para que estes também sejam renderizados na tela.

2.5 Interface Web

2.5.1 Arquitetura e tecnologias

A interface gráfica do sistema é construída sobre o framework Dash e a biblioteca Dash-Leaflet, executando sobre um servidor Flask em Python. O mapa interativo utiliza a biblioteca JavaScript Leaflet com renderização configurada sobre um elemento HTML5 Canvas, viabilizando o desenho

de milhares de pontos geográficos sem degradação do tempo de resposta da interface gráfica, mesmo durante as operações de zoom e movimento pelo mapa por parte do usuário.

2.5.2 Painel de controle

A coluna lateral esquerda organiza os controles de configuração e as ferramentas de filtragem do sistema. Durante a inicialização, uma rotina em segundo plano carrega o grafo da malha viária e instancia a KD-tree de segmentos de rua, reportando o progresso da execução por meio de um timer que consulta o estado do processo em intervalos regulares. O painel disponibiliza controles para gerar uma população sintética de entregadores, permitindo definir a quantidade de agentes que serão gerados em torno dos restaurantes. Para o catálogo de restaurantes, são disponibilizados campos que filtram os registros por bairro oficial e realizam buscas textuais por nome, razão social, endereço ou identificador numérico, sincronizando as opções de seleção com o índice de busca em memória.

2.5.3 Renderização

A área principal da tela exibe a camada base de malha geográfica do OpenStreetMap sobreposta por camadas vetoriais no padrão GeoJSON. Para controlar o custo de renderização quando há grandes volumes de pontos, os marcadores no mapa são agrupados em diferentes níveis de aproximação por meio do algoritmo SuperCluster. Estabelecimentos e entregadores são distribuídos em camadas independentes que podem ter sua viibilidade desativada. Funções em JavaScript vinculadas aos eventos do Leaflet fazem com que os dois tipos de entidade tenham marcadores diferentes: restaurantes são identificados por marcadores circulares com tons diferentes dependendo do tamanho do cluster, enquanto entregadores são denotados por marcadores em formato de losango, coloridos em laranja para posições ativas e em cinza para posições descartadas durante uma busca.

2.5.4 Interação e visualização de consultas

O usuário pode selecionar um estabelecimento tanto por meio do menu de busca textual quanto clicando sobre o marcador correspondente no mapa. Após a escolha do estabelecimento e a definição do raio de atendimento, o sistema executa a recuperação dos candidatos na trie compacta e o roteamento sobre a malha viária. A interface desenha sobre o mapa uma camada circular correspondente ao raio de atendimento, ajusta o centro e o nível de aproximação da câmera para enquadrar a área de cobertura e projeta em painéis flutuantes os dados cadastrais do local e a relação dos entregadores encontrados, detalhando a distância, o comprimento do trajeto, e o tempo estimado de viagem até o restaurante (que é calculado somando o `travel_time` de cada aresta que conecta o entregador ao restaurante pelo menor caminho, obtido por `osmnx.routing.shortest_path`).

3 Experimentos e Resultados

Nessa seção, apresentamos experimentos conduzidos para avaliar o tempo de processamento das operações principais da plataforma, para diferentes tamanhos de input. As medições foram coletadas usando `time.perf_counter_ns()`, e foi feita uma rodada completa de geração e busca espacial antes delas para que os custos de inicialização não contaminem os resultados.

3.1 Geração de Entregadores

Foi testado o tempo de processamento da geração de entregadores para diferentes quantidades N de entregadores.

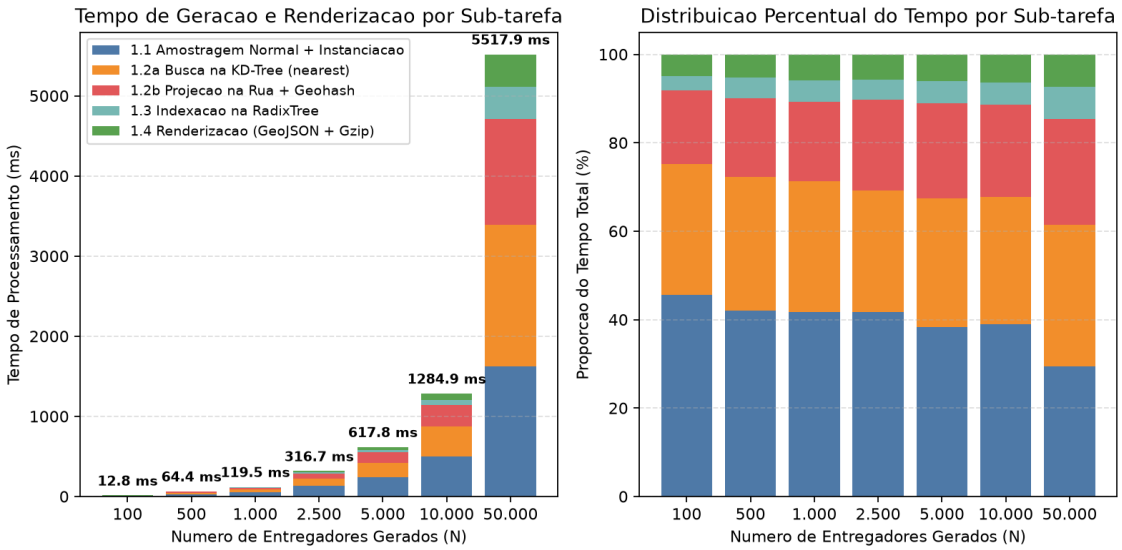


Figure 1: Distribuição dos tempos de processamento para cada sub-tarefa da geração (amostragem e instanciação, busca na KD-Tree, projeção na rua e criação de geohash, indexação na trie compacta, e renderização no mapa) para diferentes quantidades de entregadores.

Nota-se que o tempo de processamento parece crescer de forma aproximadamente linear com o valor de N, mas que certas sub-tarefas crescem mais rapidamente do que outras: à medida que N aumenta, a sub-tarefa de amostragem Normal começa a ser dominada levemente pela projeção na rua e geohashing, juntamente da indexação na trie compacta ou árvore radix.

3.2 Busca em Raio

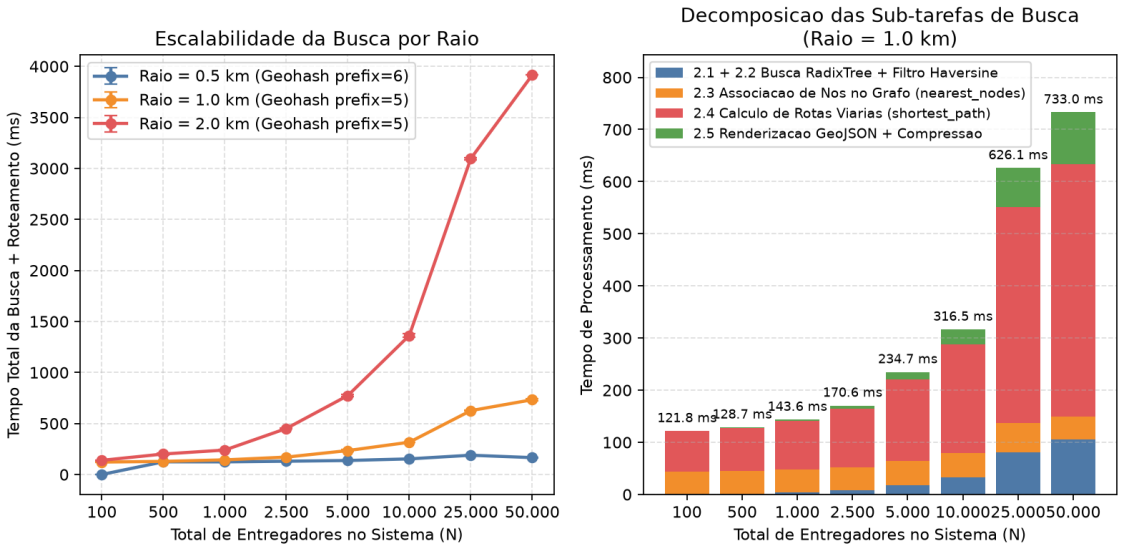


Figure 2: Distribuição dos tempos de processamento para cada sub-tarefa da busca no raio de um restaurante (busca na trie compacta, associação de nós no grafo, cálculo das rotas viárias, e renderização no mapa)

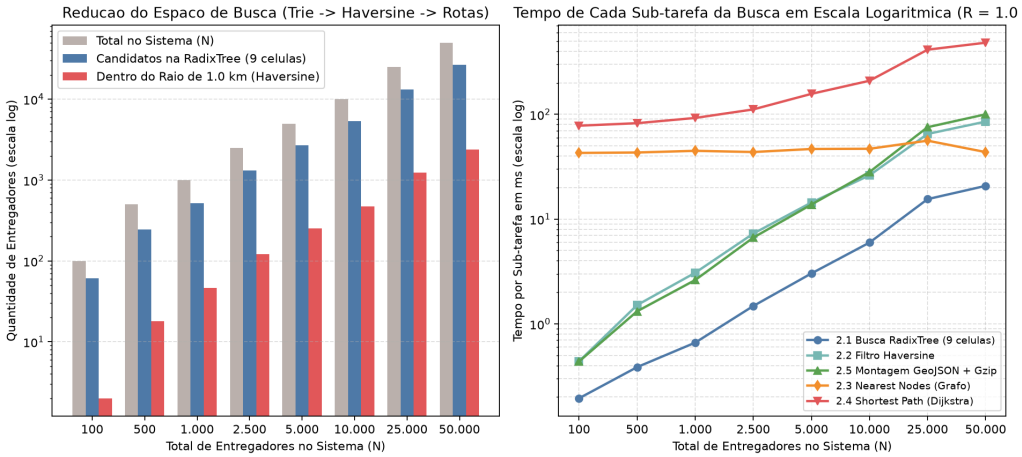


Figure 3: (Esquerda) tamanhos do espaço de busca para um raio de 1 km quando o número N de entregadores varia. (Direita) tempos de processamento para cada sub-tarefa da busca no raio de um restaurante, em escala logarítmica, para um raio de 1 km.

A consulta aos nove prefixos na árvore radix consome tempo na escala de milissegundos mesmo no volume máximo testado, descartando quase toda a base antes de processamentos geométricos mais densos. A filtragem por Haversine opera com custo linear proporcional apenas aos candidatos da vizinhança da célula, isolando, por exemplo, 474 entregadores a partir de 5.335 candidatos regionais em 26,16 ms para 10.000 entregadores em raio de 1,0 km. O cálculo dos caminhos mínimos sobre a malha viária é o maior determinante do tempo total, atingindo mais de 3 segundos na situação limite de $N = 50.000$ e raio de 2 km. Em configurações de raio de até 1 km com até 10.000 entregadores, o tempo total de resposta da consulta permanece menor que 320 ms, tornando a execução bastante viável para o uso interativo.

4 Conclusão

A interseção entre a geometria computacional e a manipulação de sequências não é imediatamente evidente, mas certos casos de uso demonstram que ela existe e é relevante. Transformar um problema que intuitivamente parece apenas geométrico em um problema de casamento de sequências leva a soluções novas e eficientes, que podem ter capacidades úteis que algoritmos puramente baseados na geometria não necessariamene compartilham. Esse Trabalho Prático demonstrou que as sub-áreas do estudo dos algoritmos contêm conexões entre si que merecem ser exploradas.

Não somente juntas como também por si só, a geometria computacional e a manipulação de sequências foram exploradas nesse trabalho. A tarefa de alinhar os entregadores à rua mais próxima, inicialmente aparentando trivial, foi um problema que demandou bastante consideração para chegar a uma solução eficiente, ainda que aproximada. A busca em raio, também, é um exemplo de etapa notável do trabalho, envolvendo tarefas interessantes como a implementação da trie compacta e a adaptação da sua busca para alcançar células adjacentes, que exigiram aprender mais profundamente sobre a estrutura de dados e considerar a eficiência de diferentes abordagens. Trabalhos Práticos como esse são úteis para incentivar que o aluno considere de fato os algoritmos que usa, e as relações ocultas entre as estruturas que manipula.

References

[1] Prefeitura de Belo Horizonte. *Cadastro de Endereços.* `https://dados.pbh.gov.br/dataset/endereco-cadastro-tecnico-municipal/resource/74dbdfce-52b4-452c-b16c-d1737191084b`

[2] Hengyi Wang. *Weiszfield algorithm.* `https://hengyiwang.github.io/Notebooks/3DV/Basics/weiszfield/`